SLAM Trainee Module 2026

IIT Bombay Racing Team

March 16, 2026

Instructions

- Trainees are required to update the following task sheet: Module Progress
- Documentation is a must for every checkpoint, and trainees are required to create a Google Doc, Notion page, or any other form of document sheet where they document their learning, errors encountered, and doubts. This doc must be made accessible to 'anyone with the link' and the link for the same must be updated on the task sheet.
- Feel free to ask questions to JDEs/DEs if you feel stuck or want to better understand a topic.
- Keep in mind that performance in modules is how the team will judge your abilities and effort to assign subsystems once the time comes.
  Have a fun learning experience!

List of Figures

1 Track layout of 'small track' in EUFS Simulator 5
2 EKF-SLAM Results on EUFS Simulator 17
3 GraphSLAM for a full lap 20
4 Graph screenshot for a full lap 20
5 FastSLAM2.0 for a full lap, real life results 22

Contents

1 Introduction to SLAM **5**
1.1 SLAM preview 6

2 Probability and Stochastic processes: Checkpoint 1 **6**
2.0.1 Introductory Resource 6

3 SLAM **7**
3.1 The Formal Problem Statement 7
3.1.1 Problem Definition 8
3.2 Solution Techniques 8
3.3 Types of SLAM - Based on Working 9

4 System Modeling - Motion Update and Introduction to Measurement Update: Checkpoint 2 **9**

5 Data Association: Checkpoint 3 **10**

6 Kalman Filters and State Estimation **11**
6.1 Extended Kalman Filters - From Kalman Filters to Extended Kalman Filters 11
6.2 State Representation: Localization and full SLAM 12
6.2.1 Motion Update 14
6.2.2 Measurement Update 14

7 Implementing Your Own EKF-Based Localization System: Checkpoint 4 **15**

8 Algorithms Developed by Our Team **16**
8.1 EKF SLAM 16
8.2 FastSLAM 1.0 17
8.3 GraphSLAM 18
8.4 FastSLAM 2.0 21
8.4.1 Core 21
8.4.2 Extensions to FastSLAM 2.0 21

1 Introduction to SLAM

Watch Introduction to SLAM (Cyrill Stachniss) to better understand the given text.
There are many occasions when an autonomous system like a robot needs to navigate in an environment that is mostly unknown to it beforehand. Navigation involves determining where the robot itself is in the environment (localization) as well as determining where and how it must go to its destination (path planning). It seems that the aspect of localization becomes impossible without having a map or model of the environment. Thus, it becomes necessary to create a map of the environment while the robot navigates through the environment.

SLAM problem is hard because it is kind of a paradox, i.e.: in order to build a map, we now need the position. To determine our position, we need a map. It is like a chicken-egg problem. In practice these two problems can be solved independently of each other. A good map is needed for localisation, while an accurate pose estimate is needed to build a map. However, techniques have been developed that enable a robot to do localization and mapping efficiently at the same time. These techniques come under the umbrella of Simultaneous Localization and Mapping, or SLAM in short.

The Figure1 below is a visualization of an autonomous race car on a typical track. As you can see in this figure, the track boundaries consist of blue ones on the left and yellow cones on the right. SLAM does not need distinct colour cones on both sides of the track; we track the colours of these cones for path planning.

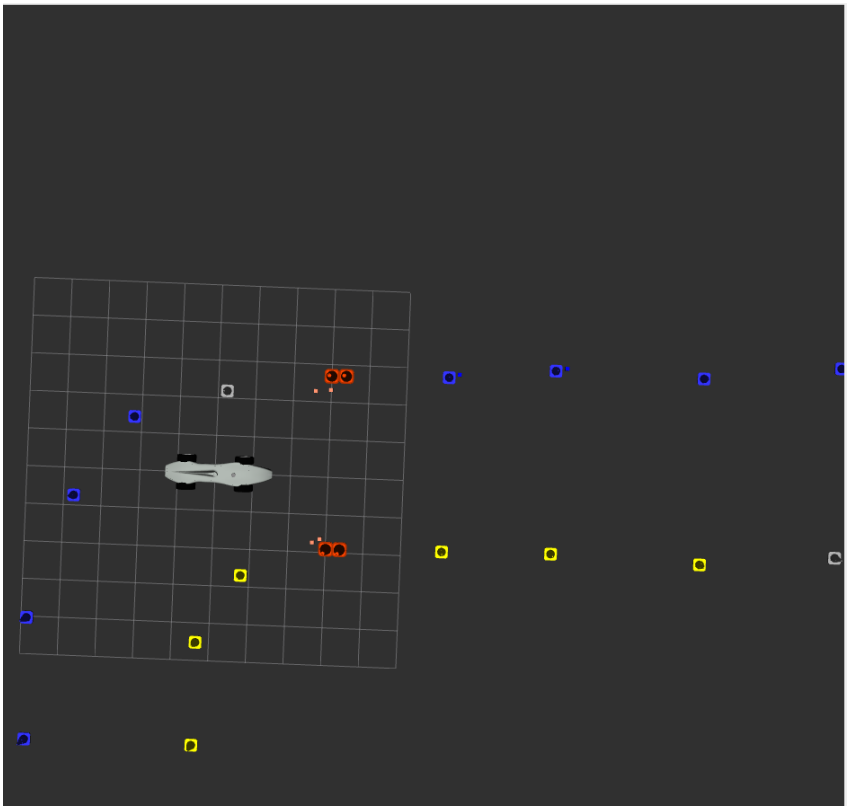

Figure 1: Track layout of 'small track' in EUFS Simulator

Chalmers' Trackdrive run at Formula Student Germany 2023

Observe this video, where the car performs SLAM in the first lap, and once it makes the map, switches to just localization in later laps, and the car goes brrrr.

1.1 SLAM preview

As discussed in the previous section, SLAM is responsible for building a map of its environment, and localizing (tracking its position) in that map.
Now what is a map? A map is any information in the environment that we (SLAM) are interested in. It is easy to guess that in our case, the information is the positions of the cones in a 2D space (grid). Therefore, for us, a map is really just a list of landmark locations, i.e., (l_x, l_y), that is, the landmark x and y coordinates in our map (remember 2D grid). As for localization, we keep a track of our vehicle's position in that virtual map (virtual because it is). We above discussed why the problem of SLAM is similar to the chicken and egg problem, that we need a known map to accurately achieve localization, and accurate localization to accurately map the environment.

How we proceed to solve SLAM, localization, or mapping problem

At every instance (discrete time instance), the vehicle receives some control inputs like throttle, steer, etc, and some measurements of the environment, which in our case, are the sensed location of the cones/landmarks in the car frame, by perception (perception gives landmark data in car frame, obviously). Our aim is to build an algorithm; at every instance that we get this data from sensors and perception, we pass this data along with previous car data SLAM has tracked/stored (vehicle poses, previously mapped landmarks, and their properties), and use this information to predict the next state (position, velocity, dynamics) of the car.

2 Probability and Stochastic processes: Checkpoint 1

Some good references for introduction to probability and statistics: To build the required background for probabilistic robotics, trainees are encouraged to review the following resources on probability theory and stochastic processes.

2.0.1 Introductory Resource

- **An Introduction to Probability Theory** `https://youtu.be/N8OZSyCvaU4?si=6yWu4X4p-3i1RNlQ`

Short Intuition-Building Videos (Steve Brunton)

- `https://youtu.be/-7QG2itL1u4?si=f-q75EZ4wqthpTLm`
- `https://youtu.be/8n1n0OM4gLk?si=V2nh2YBZ5zcM-99D`
- `https://youtu.be/h9j849vAsAA?si=pEIVBSCsE9JvfS2F`
- `https://youtu.be/NBo5bXIX7Ac?si=4eHdeYTAho5ifhyp`

- https://youtu.be/pribJ8bUBzo?si=_v2HGYX1JM1AhQiS
- https://youtu.be/dmSRMYQsM8w?si=npnWpMgC2pXOyeet
- https://youtu.be/QKPdk57y7Ck?si=0GGmkKVce9op_RaH

In-depth Lectures (Optional)

If you want to develop a deeper understanding of probability theory and stochastic processes:

- https://youtu.be/PNrqCdslGi4?si=5ZdmrMoVziUIo2mU
  (Random variables discussion begins around 34:00)
- https://youtu.be/k2BB0p8byGA?si=FCGzROutegpFCCuH
- https://youtu.be/72QjzHnYvL0?si=adpf2PfX6wkOxk43
- https://youtu.be/J70dP_AECzQ?si=WahX2Wqk8kQ3wMtT
- https://youtu.be/IujCYxtpszU?si=wvjS8q0FFUfom2RL
- https://youtu.be/MF-XSJOsGqw?si=bWev-41mi6IrizVd
  (Refer primarily to the Chi-square distribution section)

A good understanding of probability and stochastic processes is the foundation for SLAM, and thus, a good understanding of core theory is necessary. This checkpoint involves understanding the basics of probability theory and its application in SLAM theory. Content will be covered in meets, and mostly is to be studied from the reference material.

3 SLAM

3.1 The Formal Problem Statement

Given a set of control inputs at different time steps and a set of observations over time, the problem is to estimate the current state of the robot, the entire path followed by the robot, and the map of the environment. This is done in a probabilistic manner due to the existence of noise and bias in the controls as well as sensor observations.

The set of observations and the set of control commands till time t are collectively represented as $Z_{1:t}$ and $U_{1:t}$, respectively.

The Simultaneous Localization and Mapping (SLAM) problem is a fundamental task in robotics that involves constructing a map of an unknown environment while simultaneously estimating the robot's pose within that map. It can be mathematically defined as follows:

3.1.1 Problem Definition

Given:

- A sequence of robot control inputs: $U_{1:T} = \{u_1, u_2, \ldots, u_T\}$, where u_t represents the control input at time step t.
- A sequence of robot sensor measurements: $Z_{1:T} = \{z_1, z_2, \ldots, z_T\}$, where z_t represents the sensor measurement at time step t.
- An initial estimate of the robot's pose: x_0.
- An initial estimate of the map of the environment: m_0.

The goal of SLAM is to estimate the robot's trajectory over time:

$$X_{1:T} = \{x_1, x_2, \ldots, x_T\} \tag{1}$$

and to build a map of the environment m by optimizing the joint posterior distribution:

$$P(X_{1:T}, m | Z_{1:T}, U_{1:T}) \tag{2}$$

This joint distribution represents the probability of the robot's trajectory and the map given the sensor measurements and control inputs. The SLAM problem involves finding the maximum a posteriori (MAP) estimate of this joint distribution.

3.2 Solution Techniques

To solve the SLAM problem, various techniques and algorithms are used, including:

- **Extended Kalman Filter (EKF) SLAM** - Filter-based SLAM algorithms, such as Extended Kalman Filter (EKF) SLAM and its variants, use recursive filtering techniques to estimate the robot's pose and update the map incrementally. These algorithms maintain a belief over the robot's pose and map based on the sensor measurements and control inputs. EKF SLAM is well-suited for environments with mild nonlinear dynamics and Gaussian noise assumptions, but it can struggle with strong nonlinearities and large uncertainties.
- **GraphSLAM** - GraphSLAM and its variants, treat the SLAM problem as a nonlinear optimization problem. They formulate the SLAM problem as a graph, where the nodes represent robot poses and landmarks, and the edges represent the constraints between them. By optimizing the graph using techniques like least squares or non-linear optimization methods, these algorithms jointly estimate the robot's trajectory and the map. Graph-based SLAM can handle nonlinearities and non-Gaussian noise better than filter-based approaches.
- **Particle Filter SLAM** - Particle Filter SLAM, also known as Monte Carlo SLAM or Rao-Blackwellized Particle Filter (RBPF) SLAM, combines the benefits of particle filters with the SLAM problem. It uses a set of particles to represent the belief over the robot's pose and map. Particle filters allow non-Gaussian representations and handle nonlinearities effectively. Particle Filter SLAM algorithms use resampling and importance sampling techniques to update the particles based on the sensor measurements and control inputs, estimating the robot's trajectory and constructing the map.

These algorithms employ probabilistic approaches, such as the Kalman filter or particle filters, to estimate the robot's pose and update the map based on sensor measurements and control inputs.

3.3 Types of SLAM - Based on Working

Full SLAM involves estimating the entire path of the robot after all the sensor observations are collected and all control commands till time T are executed. This can be mathematically expressed as determining the probability distribution:

$$p(x_{1:T}, m \mid z_{1:T}, u_{1:T}) \tag{3}$$

where $x_{1:T}$ represents the trajectory of the robot (position and direction of movement), and m represents the map of the environment. This approach takes into account all the sensor measurements and control inputs available up to time step T. Full SLAM is useful when the entire dataset is available, allowing for more accurate and globally consistent estimates of the robot's trajectory and the environment.

On the other hand, Online SLAM involves estimating the current state of the robot based on the sensor observations and control commands up to the current time t. This can be expressed mathematically as determining:

$$p(x_t, m \mid z_{1:t}, u_{1:t}) \tag{4}$$

In many real-time applications, Online SLAM is more relevant. It represents the probability distribution that captures the updated knowledge about the robot's pose at time step t and the map, given all the sensor measurements and control inputs up to that point. This approach is suitable for real-time applications where the robot needs to continuously update its estimate of the environment while operating.

Which one do you think would be more suitable for our case? Full SLAM or Online SLAM?

4 System Modeling - Motion Update and Introduction to Measurement Update: Checkpoint 2

PREREQUISITE: Make sure you are familiar with the concept of motion update and measurement update we covered in the meeting.

Objective In this checkpoint, you will create a ROS package and develop a C++ node for motion update. The task involves setting up the required dependencies, subscribing to relevant topics, and visualizing data in RViz. Additionally, you are supposed to think of some methods to perform the measurement update, and try to express them in the form of pseudo-code, or, if possible, try to code them.

Steps to Follow

1. **Create a ROS Package**
   - Initialize a new ROS package.
   - Add a C++ file that will serve as the localization node.
   - Update `package.xml` and `CMakeLists.txt` with the required dependencies.

2. **Define Subscribed Topics**
   - Identify the topics necessary for the **motion update** (recall the input to slam as told in the meeting).
   - Specify the message types for each topic.
3. **Visualization in RViz**
   - Display the ground truth map.
   - Visualize the car's ground truth pose using an arrowhead marker.
4. **Motion Update**
   - Identify the state variables and matrices needed for motion update.
   - Create an appropriate callback for motion update that gets called every time you receive a message from the wheel speed sensors of the car, and then perform the required calculations.
   - Visualize your motion update-based predicted pose of the car.

Expected Outcome: By the end of this checkpoint, you should have a functional ROS package that subscribes to necessary topics, handles motion updates, and visualizes the car's position in RViz.

5 Data Association: Checkpoint 3

PREREQUISITE: Make sure you are familiar with the concept of data association, and you have studied and understood NN data association, KD-Trees, and ML+IC (Maximum Likelihood and Individual Compatibility) covered in the meeting. Given:

- The car's state at time t is given as:

$$[x,\, y,\, \psi]$$

  where (x, y) and yaw(ψ) represent the car's position and orientation.

- A set of sensor readings is provided as:

$$[(r_1,\, \theta_1),\, (r_2,\, \theta_2),\, \ldots,\, (r_m,\, \theta_m)]$$

  where r_j is the range and θ_j is the bearing of the j-th reading.

- A known map is provided as:

$$[(x_1,\, y_1), (x_2,\, y_2,\,) \ldots,\, (x_n,\, y_n)]$$

  where (x_i, y_i) for $i = 1, 2, \ldots, n$ are the positions of n landmarks. *Note: The data structure might vary based on implementation, but the amount and meaning(interpretation) of data will be same.*

Task 3.1: Write a C++ function that:

1. Performs nearest neighbors data association on this input data.
2. Returns an array of indices, where each index corresponds to the associated landmark for the reading (by providing the landmark's exact index in the state), or -1 indicating a new landmark.

Task 3.2: Implement an ML+IC (Maximum Likelihood and Individual compatibility) function given the above input data, and return the output in the above given output format.

Task 3.3: Implement KD-Trees for search in the above function (or you can directly implement KD-Trees from the start)

6 Kalman Filters and State Estimation

Among the earliest approaches is the utilization of Kalman filters and extended Kalman filters to solve SLAM. Kalman filter is based on Recursive state estimation- The Bayes filter.

Note before you start with the following resources*: Don't be scared of the length of these vidoes, the amount of content covered, or how lengthy or complete the research paper is. You can always slow down, use other resources, and LLMS to understand some subparts of the topics listed below. You can always ask us for help if nothing helps (or you just want our help)!*

Links to resources

- **A easy to understand paper on Kalman Filters** - Understanding the Basis of the Kalman Filter Via a Simple and Intuitive Derivation
- For those interested in the original paper by RE Kalman, 1960
- Kalman Filter
- *Recursive state estimation- Bayes filter.*
- Kalman Filter & EKF (Cyrill Stachniss) (watch till 43:00)

6.1 Extended Kalman Filters - From Kalman Filters to Extended Kalman Filters

The Kalman Filter is one of the most fundamental algorithms used for **state estimation**. It is a recursive algorithm that estimates the state of a dynamic system from noisy measurements. At every time step, the filter maintains a **belief** about the system's state in the form of a Gaussian distribution, characterized by a **mean** (state estimate) and a **covariance** (uncertainty).

The classical Kalman Filter assumes two important conditions:

- The **motion model** describing how the state evolves over time is **linear**.
- The **measurement model** relating the state to observations is also **linear**.

Under these assumptions, the Kalman Filter provides an optimal solution for estimating the system's state when the noise is Gaussian.

However, in most real-world robotics problems, including SLAM, the system dynamics and observation models are **nonlinear**. For example, a robot's motion often involves trigonometric relationships (such as sin and cos), and sensor observations like range and bearing are inherently nonlinear functions of the robot's state.

To handle such nonlinearities, the **Extended Kalman Filter (EKF)** extends the classical Kalman Filter by **locally linearizing** the nonlinear models. This is done by computing the **Jacobians** of the motion and measurement functions around the current estimate.

In other words:

Kalman Filter = Linear models
Extended Kalman Filter = Nonlinear models approximated as linear locally

By linearizing the system at every step, the EKF allows us to apply the Kalman Filter framework to nonlinear robotic systems such as localization and SLAM.

In the following sections, we will use EKF to implement a recursive state estimator consisting of two steps: **motion update (prediction)** and **measurement update (correction)**.

6.2 State Representation: Localization and full SLAM

A crucial concept in probabilistic state estimation is the **state vector**. The state vector contains all the variables that the algorithm is trying to estimate.

Depending on whether we are solving a **localization problem** or a full **SLAM problem**, the contents of the state vector are different.

Case 1: Localization Only (Known Map)

In the localization problem, the map of the environment is already known beforehand. The robot only needs to estimate its own pose relative to this map.

Therefore, the state vector contains only the robot pose:

$$x = \begin{bmatrix} x \\ y \\ \psi \end{bmatrix}$$

where

- x, y represent the position of the robot in the global frame
- ψ represents the robot's orientation (yaw)

The landmarks are not part of the state because their positions are already known and stored in the map.

The covariance matrix therefore only represents uncertainty in the robot pose:

$$P = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} & \sigma_{x\psi} \\ \sigma_{yx} & \sigma_{yy} & \sigma_{y\psi} \\ \sigma_{\psi x} & \sigma_{\psi y} & \sigma_{\psi\psi} \end{bmatrix}$$

During the measurement update, observations are compared with the known landmark positions in the map to correct the robot's pose estimate.

Example in our system: After completing one lap and building the map, the car can use EKF localization in later laps to estimate its pose relative to the known cone map.

Case 2: Full SLAM (Unknown Map)

In the SLAM problem, the robot does not know the map beforehand. It must estimate both:

- its own pose
- the positions of all landmarks

Therefore, the state vector must include both the robot pose and landmark positions. If the robot observes N landmarks, the state vector becomes:

$$x = \begin{bmatrix} x_r \\ y_r \\ \psi_r \\ l_{1x} \\ l_{1y} \\ l_{2x} \\ l_{2y} \\ \vdots \\ l_{Nx} \\ l_{Ny} \end{bmatrix}$$

where

- x_r, y_r, ψ_r represent the robot pose
- (l_{ix}, l_{iy}) represent the coordinates of the ith landmark

The covariance matrix now contains uncertainty for both the robot and landmarks:

$$P = \begin{bmatrix} P_{rr} & P_{rm} \\ P_{mr} & P_{mm} \end{bmatrix}$$

where

- P_{rr} represents uncertainty in the robot pose
- P_{mm} represents uncertainty in landmark positions
- P_{rm} represents correlations between robot pose and landmarks

These correlations are important because observing a landmark updates the uncertainty of both the robot and the landmark simultaneously.

State Size Growth

If there are N landmarks in the map, the dimension of the state vector becomes:

$$3 + 2N$$

This means the covariance matrix grows as:

$$(3 + 2N) \times (3 + 2N)$$

Because of this quadratic growth in the covariance matrix, EKF-SLAM becomes computationally expensive as the number of landmarks increases.

Summary

Problem Type	State Vector	Map Knowledge
Localization	Robot pose only	Map known
SLAM	Robot pose + landmarks	Map unknown

Our implementation of EKF is divided into two parts (modified to suit our setup)

6.2.1 Motion Update

The motion update step predicts the robot's new pose based on control inputs, such as velocity and angular rotation. It uses a motion model to estimate the expected change in the robot's pose. The motion model captures the kinematics or dynamics of the robot and takes into account factors like wheel odometry or inertial measurements.

During the motion update, the algorithm propagates the current estimate of the robot's pose by incorporating the control inputs. It predicts the new position and orientation of the robot based on the motion model. This prediction provides an initial estimate for the next step. It is expressed as:

$$p(x_t|x_{t-1}, u_t)$$

where:

- $p(x_t|x_{t-1}, u_t)$ denotes the conditional probability distribution of the state variable x_t at time t, given the previous state x_{t-1} and the control variable u_t at time t.

6.2.2 Measurement Update

The measurement update step corrects the predicted pose estimate using sensor measurements. These measurements can be obtained from various sensors such as cameras, lidar, or range finders. The measurements capture the robot's perception of the environment, such as landmarks or features.

In the measurement update, the algorithm compares the predicted pose with the actual sensor measurements. It computes the likelihood or the probability of the measurements given the predicted pose. This likelihood is used to update the pose estimate using the Kalman filter equations. The update improves the accuracy of the estimated pose and allows for the creation of a map by associating the measurements with landmarks in the environment.

It refers to the model describing the expected observations given the current state of the robot. It is expressed as:

$$p(z_t|x_t)$$

where:

- $p(z_t|x_t)$ denotes the conditional probability distribution of the observation variable z_t at time t, given the state variable x_t at the same time.

Some examples include the Laser range bearing sensor model and the Beam end-point model.

7 Implementing Your Own EKF-Based Localization System: Checkpoint 4

We know your head is probably spinning after tackling all these complex terms and intense math, but trust us, it's all hands-on coding from here on out. No more theory, just action. Keep your chin up and get ready to dive in.

Prerequisites:

- Fundamentals of ROS2 Humble (subscribing and publishing to topics, listing topics, playing rosbags, and RViz visualizations)
- Understanding of EKF Localization

We will develop an EKF-based localization ROS node, applying the theory you've learned. The algorithm's operation will be visualized in real-time using RViz. Since this is a substantial task, we will break it down into smaller checkpoints for better progress tracking.

Issue with Raw Odometry: As you have observed, relying solely on raw odometry data leads to error accumulation, causing the car's estimated pose to drift over time. To mitigate this, we will implement a **measurement update** function that utilizes observation information to correct the car's pose.

Tasks to Complete

1. **Implement a Measurement Update Function**
   - Create a callback function to process observation data.
   - Compute the Kalman gain based on sensor observations.
   - Update the car's estimated pose using the Kalman filter equations.
2. **Addressing Data Association**
   - Observation data must be correctly matched to known map features.
   - Investigate **data association** techniques to determine which cones detected by perception correspond to those in the map.
   - Research and identify methods to effectively solve the data association problem.

Expected Outcome By implementing measurement updates, the car's estimated pose should be corrected using sensor observations, reducing divergence over time. Understanding and solving the data association problem will further refine localization accuracy.

An exercise

A key concept in probabilistic robotics is that the robot's pose and landmarks are not represented as exact values but as **probability distributions**. In EKF, this uncertainty is represented using the covariance matrix.

For a 2D variable such as a landmark position (x, y), the covariance matrix can be visualized as an **uncertainty ellipse**. The size and orientation of this ellipse indicate how uncertain the estimate is.

Assignment: Visualizing Uncertainty

Your task is to visualize uncertainty during localization and mapping.

- Extract the covariance matrix of the robot pose or landmarks.
- Convert the covariance into a 2D ellipse representation.
- Visualize these ellipses in **RViz** or using a plotting library such as `matplotlib`.

Note your observation on how covariances change between motion updates, between measurement updates, and try to analyze them and discuss with others.

Think About

- What happens to the robot's uncertainty if no measurements are received for a long time?
- How does the uncertainty change when the robot observes multiple landmarks?
- Why does data association become harder when uncertainty grows?

Understanding uncertainty visualization will help you debug SLAM algorithms and interpret the behavior of probabilistic estimators.

8 Algorithms Developed by Our Team

8.1 EKF SLAM

Implementing EKF-SLAM required a significant team effort, as developing and testing any SLAM algorithm from scratch is no trivial task. We largely followed Le Large's thesis [1] to structure our implementation and referred to the SLAM Course Paper by Joan Sol 'a [2] for the function responsible for initializing new landmarks. Our code was built and tested on the EUFS simulator; however, its performance did not meet our expectations. Now, it is time to critically assess the inherent limitations of EKF-SLAM.

[1] Le Large's Thesis
[2] SLAM Course Paper by Joan Solà

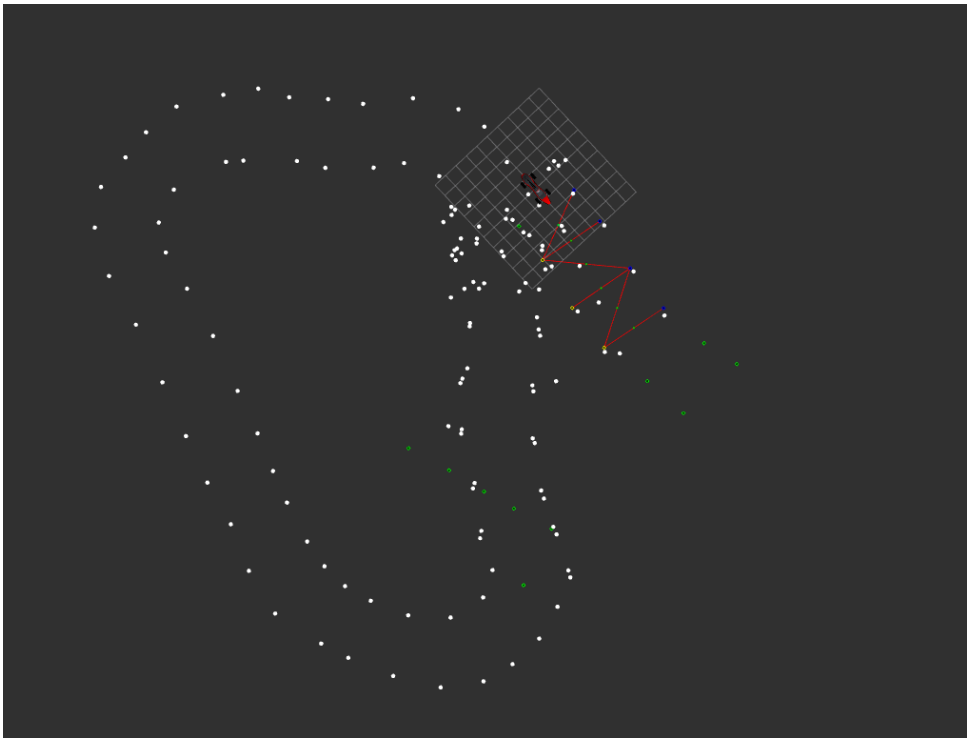

Figure 2: EKF-SLAM Results on EUFS Simulator

8.2 FastSLAM 1.0

FastSLAM 1.0 is a technique that allows a robot to build a map of its environment while simultaneously determining its own location. While you're already familiar with the Extended Kalman Filter (EKF) for tracking landmarks, FastSLAM extends these ideas by managing uncertainty in the robot's path using a particle filter.

How FastSLAM Works

1. **Splitting the Problem:** Traditional SLAM requires estimating both the robot's path and the map as one problem. FastSLAM splits this into:
   - **Robot Trajectory:** It uses a set of hypotheses (called *particles*) to represent different possible paths the robot might have taken.
   - **Landmark Mapping:** Each particle carries its own set of EKFs one for each landmark to update and maintain its positions.
2. **Math Behind it:**
   The overall SLAM problem is expressed probabilistically as:

$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t}) = p(x_{1:t} \mid z_{1:t}, u_{1:t}) \prod_j p(m_j \mid x_{1:t}, z_{1:t}, u_{1:t})$$

   where:

   - $x_{1:t}$ is the robot's trajectory.
   - m_j represents each landmark's position.

- $z_{1:t}$ and $u_{1:t}$ are the sensor measurements and control inputs.

This means FastSLAM tracks multiple possible trajectories using particles, with each particle independently updating landmarks using EKF.

The Role of Resampling

Why Resampling?
In a particle filter, each particle is assigned a weight based on how well it explains the sensor data. Over time, some particles become more likely than others.

How It Works:
Resampling generates a new set of particles by favoring those with higher weights, while less likely particles are discarded. This maintains a diverse yet focused set of hypotheses, ensuring robust path estimation even in uncertain conditions.

Key Benefits

- **Scalability:** By treating the map as several smaller EKF problems, FastSLAM avoids the heavy computational cost of maintaining a single large covariance matrix.
- **Flexibility in Uncertainty:** The use of particles enables the management of multiple hypotheses about the robot's path, which is particularly useful in noisy or complex environments.
- **Real-World Performance:** Combining EKF updates for landmarks with particle filtering for the trajectory provides a practical and robust solution for mapping tasks.

FastSLAM2.0 exists as well with major improvements in how the resampling step is performed and how the posterior is calculated. However, It is yet to be developed by the team and involves a much more complex implementation. You are encouraged to read about FastSLAM [3] and Particle Filters[4]

8.3 GraphSLAM

GraphSLAM formulates the **SLAM problem** as a **global optimization** problem: the robot's motion and landmarks are represented as **graph nodes** along with the **constraints** defined by edges between them, and each edge has an **information matrix**, which intuitively **weighs the edge** during optimization.

It works by **creating** and **incrementally optimizing** this graph, providing a configuration of **poses** and **landmarks** that most likely fit with the observed data. However, unlike **FastSLAM**, which uses **particle filters** and separate **EKFs**, it solves for the **full trajectory and map together**—making it suitable for **larger mapping problems**.

We have 2 different implementations:- First is **graph based optimization** which uses the **g2o library**. **g2o** is a widely used library designed for **graph optimization**, achieving efficiency by utilizing the **sparse nature** of the underlying matrices.

[3] FastSLAM by Montemerlo
[4] Particle Filters in Robotics

The second approach uses **Ceres Solver**, a general-purpose **nonlinear least-squares optimization** library developed and open-sourced by Google. Unlike g2o, which is specifically designed around graph representations, Ceres focuses on providing a flexible framework for defining optimization problems through **cost functions** and **residual blocks (error blocks)**. The solver then minimizes the **sum of squared errors** using advanced optimization algorithms.

The current implementation of **GraphSLAM** runs **parallelly** on **three distinct threads**:

1. **Motion update**: Every time an **odometry measurement** is received (every **5 ms**), a node corresponding to the **car pose** is added using the **bicycle motion model**. This node is connected to the **previous node** by an **edge** corresponding to the odometry measurement, with the **information matrix** of the edge equal to the **inverse of the covariance** of that particular odometry measurement.

2. On a **second thread**, every time a **set of observations** is received, a node corresponding to each **new landmark** is created. All of these and the **associated ones** are connected to the current **car pose** with edges corresponding to the **measurement**, and an **information matrix** equal to the **inverse of the covariance** of the observation.

3. **Optimization** is run **parallelly** on a **third thread**, ensuring that the **map is updated** roughly every **400 ms**, approximately once every **15 sets of observations** are received.

This **multi-threaded structure** ensures that during **data association or optimization**, **motion update** always has a thread to execute, ensuring the **latest car pose availability** for **PPC**.

As the **graph size increases**, **optimization** takes **significant time**, leading to several challenges. One major issue is that since the **map cannot be updated during optimization**, a significant amount of **observations and measurements are lost**. To handle this, the **edges and vertices** are **stored in a buffer** while optimization is in progress and then added to the graph before the next optimization cycle begins.

Additionally, to manage the **computation time**, a **sliding window** approach has been implemented, considering only a **fixed number of recent poses** for optimization. This **reduces the optimization time** at the **cost of reduced accuracy**.

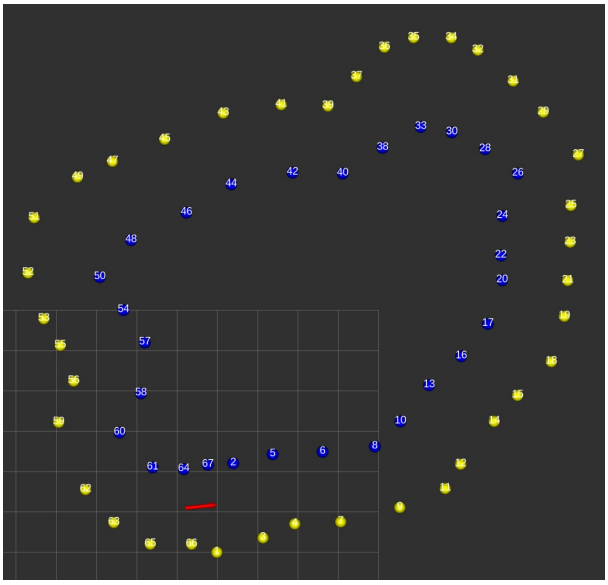

Figure 3: GraphSLAM for a full lap

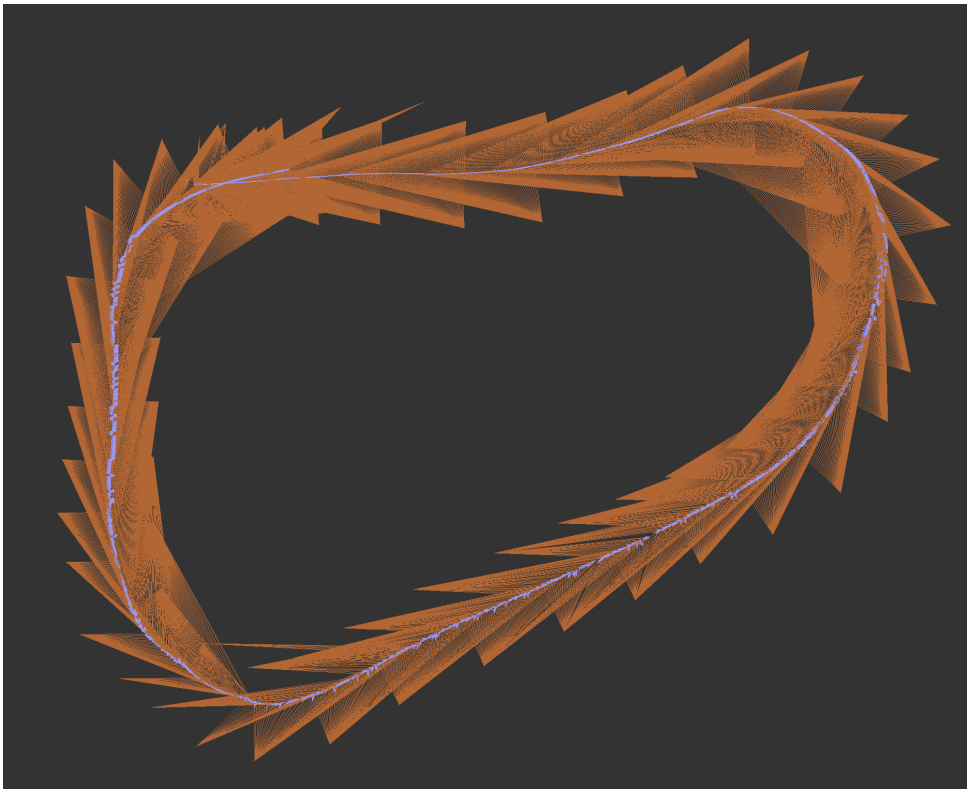

Figure 4: Graph screenshot for a full lap

8.4 FastSLAM 2.0

8.4.1 Core

FastSLAM 2.0 improves upon FastSLAM 1.0 by using a more informed proposal distribution for sampling particles. Instead of generating particles purely from the motion model, FastSLAM 2.0 incorporates the most recent sensor measurements while sampling new particles. This results in particles that are closer to the true posterior distribution, improving accuracy and reducing the number of particles required.

The key idea is to sample from a proposal distribution of the form:

$$p(x_t \mid x_{t-1}, u_t, z_t)$$

which conditions the particle generation on both the control input and the latest observation. As a result, FastSLAM 2.0 significantly reduces particle degeneracy and provides better estimation of the robot trajectory compared to FastSLAM 1.0.

8.4.2 Extensions to FastSLAM 2.0

Several extensions to FastSLAM 2.0 have been proposed in literature to improve efficiency, robustness, and scalability in complex environments. These improvements mainly focus on better particle management, improved data association, and optimization-based refinement of particle hypotheses.

KLD-Sampling

One such improvement is **KLD-Sampling**, which dynamically adjusts the number of particles used by the particle filter based on the **Kullback–Leibler Divergence (KL Divergence)** between the estimated and true distributions. Instead of maintaining a fixed number of particles, the algorithm increases the particle count when the system uncertainty grows and reduces it when the state estimate becomes more confident. This allows computational resources to be used more efficiently.

FC&ASD-PSO

An important extension to FastSLAM 2.0 introduces a hybrid optimization method called **FC&ASD-PSO** (Fractional Calculus and Alpha-Stable Distribution based Particle Swarm Optimization).

In the standard FastSLAM 2.0 framework, the process noise covariance and observation noise covariance are assumed to be known and constant. However, in real-world systems these noise parameters are often unknown or change over time, which can significantly reduce estimation accuracy.

The FC&ASD-PSO based approach adds an additional optimization step to the FastSLAM 2.0 pipeline to adaptively estimate these noise parameters. It uses a modified particle swarm optimization algorithm that incorporates fractional calculus for improved convergence and alpha-stable distributions to better explore the search space and avoid local optima.

By dynamically adjusting the noise statistics during operation, this extension improves the robustness and accuracy of FastSLAM 2.0 in environments with uncertain or time-varying sensor noise.

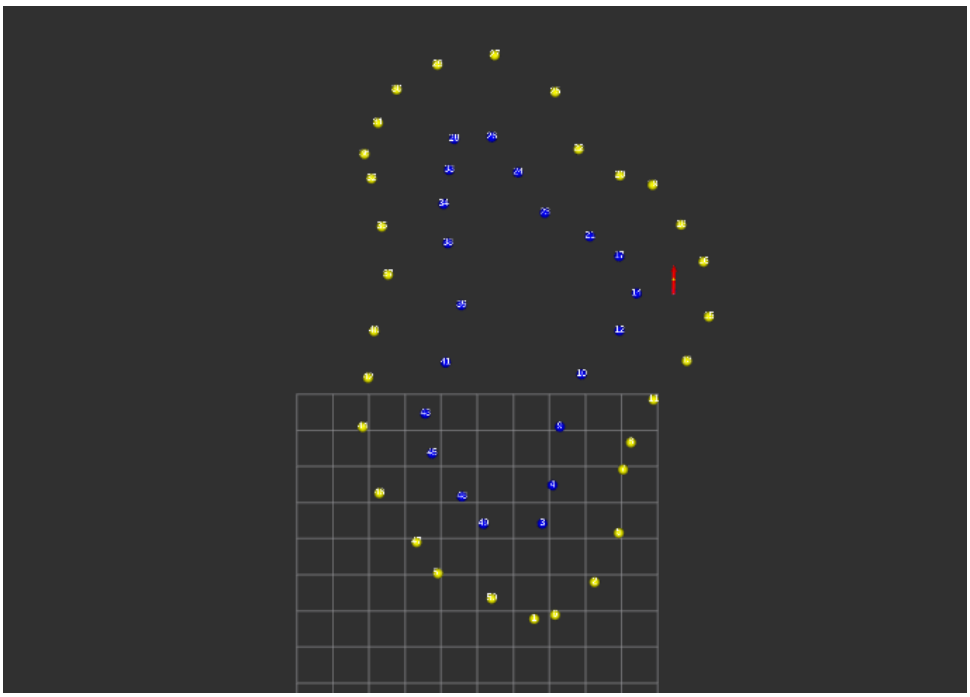

Figure 5: FastSLAM2.0 for a full lap, real life results